

目录

1. 帧总体结构 1

2. 字段定义 1

 2.1 Byte 0 — FLAGS (标志字节) 1

 2.2 Byte 1 — SEQ / CMD 2

 2.3 Byte 2 — LENGTH (有效载荷长度) 2

 2.4 Byte 3 — FRAG_INFO (分片信息) 2

 2.5 Byte 4 ~ 148 — PAYLOAD (有效载荷) 2

 2.6 Byte 149 — CHECKSUM (校验字节) 3

3. 典型帧示例 3

 3.1 图传数据帧 (无分片) 3

 3.2 紧急文字命令帧 3

 3.3 图传分片帧 (第 2/3 片) 3

 3.4 心跳 / ACK 控制帧 3

4. 处理规则 4

 4.1 发送端 4

 4.2 接收端 4

 4.3 分片重组 4

附录 A: 文字命令码表 (CMD) 4

附录 B: 控制帧类型表 (CTRL_TYPE) 4

版本: v1.0
日期: 2026-02-09
帧长度: 固定 150 字节

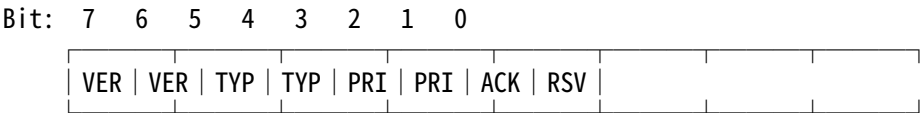
1. 帧总体结构

偏移	字段名	长度	说明
Byte 0	FLAGS	1 字节	标志字节 (版本、类型、优先级、ACK)
Byte 1	SEQ / CMD	1 字节	帧序号 (图传) 或命令码 (文字)
Byte 2	LENGTH	1 字节	有效载荷长度 (0~145)
Byte 3	FRAG_INFO	1 字节	分片信息
Byte 4 ~ 148	PAYLOAD	0~145 字节	有效载荷数据
Byte 149	CHECKSUM	1 字节	XOR / CRC-8 校验

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4 ~ 148	Byte 149		
FLAGS	SEQ/CMD	LENGTH	FRAG_INFO	PAYLOAD	CHECKSUM		
(1B)	(1B)	(1B)	(1B)	(0~145B)	(1B)		

2. 字段定义

2.1 Byte 0 — FLAGS (标志字节)



位段	位域	名称	说明
VER	[7:6]	协议版本	00 = v1, 01 = v2, 10 = v3, 11 = v4
TYP	[5:4]	数据类型	00 = 图传数据, 01 = 文字/命令, 10 = 控制帧, 11 = 保留
PRI	[3:2]	优先级	00 = 普通, 01 = 高, 10 = 紧急, 11 = 保留
ACK	[1]	应答请求	1 = 需要对方回复 ACK, 0 = 无需应答
RSV	[0]	保留位	固定为 0

2.2 Byte 1 — SEQ / CMD

此字节含义取决于 TYP 字段：

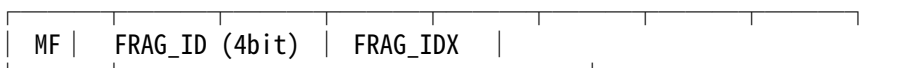
TYP 值	字段名	范围	说明
00 图传	SEQ (帧序号)	0-255	循环递增，用于检测丢帧和乱序
01 文字	CMD (命令码)	0-255	预定义消息编号（见附录 A）
10 控制	CTRL_TYPE (控制类型)	0-255	控制帧子类型（见附录 B）

2.3 Byte 2 — LENGTH (有效载荷长度)

字段	范围	说明
LENGTH	0-145	PAYLOAD 区域的实际有效字节数。超过 145 的值视为非法帧，接收端应丢弃

2.4 Byte 3 — FRAG_INFO (分片信息)

Bit: 7 6 5 4 3 2 1 0



位段	位域	名称	说明
MF	[7]	More Fragments	1 = 后续还有分片, 0 = 最后一片或未分片
FRAG_ID	[6:3]	分片组 ID	0-15, 标识属于同一批拆分数据
FRAG_IDX	[2:0]	分片序号	0-7, 标识本片在组内的顺序

未使用分片时，整个字节置 0x00。
最大支持 8 片/组 × 145 字节/片 = **1160 字节** 的完整数据。

2.5 Byte 4 ~ 148 — PAYLOAD (有效载荷)

TYP 值	PAYLOAD 内容
00 图传	原始或压缩图像数据
01 文字	UTF-8 编码文本（CMD 隐含语义时可省略，LENGTH=0）
10 控制	控制数据（如 ACK 帧中携带被确认的 SEQ）

当 LENGTH < 145 时，PAYLOAD 中 LENGTH 之后的字节应填充 0x00。

2.6 Byte 149 — CHECKSUM（校验字节）

对 Byte 0 ~ Byte 148（共 149 字节）进行 **逐字节异或（XOR）** 运算：

```
uint8_t calc_checksum(const uint8_t *frame, uint8_t len) {
    uint8_t cksum = 0;
    for (uint8_t i = 0; i < len; i++) {
        cksum ^= frame[i];
    }
    return cksum;
}
```

3. 典型帧示例

3.1 图传数据帧（无分片）

Byte 0: 0b 00_00_00_0_0 = 0x00 (v1, 图传, 普通优先级, 无需ACK)
 Byte 1: 0x1A (帧序号 26)
 Byte 2: 0x91 (载荷长度 145)
 Byte 3: 0x00 (无分片)
 Byte 4~148: [145 字节图像数据]
 Byte 149: [CHECKSUM]

3.2 紧急文字命令帧

Byte 0: 0b 00_01_10_1_0 = 0x1A (v1, 文字, 紧急, 需ACK)
 Byte 1: 0x01 (CMD=0x01: 摩擦轮卡住)
 Byte 2: 0x00 (无额外载荷)
 Byte 3: 0x00 (无分片)
 Byte 4~148: 0x00 填充
 Byte 149: [CHECKSUM]

3.3 图传分片帧（第 2/3 片）

Byte 0: 0b 00_00_00_0_0 = 0x00 (v1, 图传, 普通)
 Byte 1: 0x1B (帧序号 27)
 Byte 2: 0x91 (载荷 145 字节)
 Byte 3: 0b 1_0011_001 = 0x99 (MF=1, FRAG_ID=3, FRAG_IDX=1 第2片)
 Byte 4~148: [图像数据第 2 片]
 Byte 149: [CHECKSUM]

3.4 心跳 / ACK 控制帧

Byte 0: 0b 00_10_00_0_0 = 0x20 (v1, 控制帧, 普通)
 Byte 1: 0x01 (CTRL_TYPE=0x01: ACK)
 Byte 2: 0x01 (载荷 1 字节)
 Byte 3: 0x00 (无分片)
 Byte 4: 0x1A (确认收到 SEQ=0x1A 的帧)

Byte 5~148: 0x00 填充
Byte 149: [CHECKSUM]

4. 处理规则

4.1 发送端

1. 按帧结构填充 Byte 0 ~ Byte 148
2. 计算 CHECKSUM 写入 Byte 149
3. 若 ACK=1, 发送后启动重传定时器
4. 若定时器超时未收到 ACK, 重传该帧 (最多重传 3 次)

4.2 接收端

1. 校验 CHECKSUM, 不匹配则丢弃
2. 校验 LENGTH $\leq$ 145, 不满足则丢弃
3. 校验 VER 是否为支持的版本
4. 根据 TYP 分发至对应处理模块
5. 若 ACK=1, 回复控制帧 (CTRL_TYPE=0x01)
6. 若 MF=1 或 FRAG_IDX > 0, 进入分片重组流程

4.3 分片重组

1. 按 FRAG_ID 归组, 按 FRAG_IDX 排序
 2. 收到 MF=0 的末片后, 检查 0 ~ FRAG_IDX 是否齐全
 3. 齐全则拼接 PAYLOAD 交付上层; 缺失则等待或请求重传
 4. 重组超时时间建议: 500ms
-

附录 A: 文字命令码表 (CMD)

CMD	含义	优先级建议
0x00	保留 (空操作)	—
0x01	摩擦轮卡住	紧急
0x02 ~ 0xFF	待定义	—

附录 B: 控制帧类型表 (CTRL_TYPE)

CTRL_TYPE	含义	PAYLOAD
0x00	心跳 (Heartbeat)	无
0x01	应答 (ACK)	1 字节: 被确认帧的 SEQ
0x02	否定应答 (NACK)	1 字节: 请求重传的 SEQ
0x03	重置 (RESET)	无
0x04 ~ 0xFF	待定义	—